

DP Computer Science: B2.1 Programming Fundamentals (Python)

Overview

This section explores the foundational elements of programming in Python, including variables, data types, string manipulation, exception handling, and debugging techniques. Understanding these concepts is crucial for writing effective and robust programs.

Learning Objectives

By the end of this section, students will be able to:

- Define and differentiate between **global and local variables**
- Trace** the execution of programs involving variables
- Identify and explain the roles of common data types (**Boolean, char, integer, string**)
- Implement variables and data types in basic Python programs

Topic Breakdown

Topic	Key Concepts
B2.1.1 Variables and Data Types	Global vs. local variables, data types (Boolean, char, integer, string), tracing program execution
B2.1.2 String Manipulation	Concatenation, slicing, replacement, substring extraction
B2.1.3 Exception Handling	<code>try-except</code> blocks, common exceptions, error messages
B2.1.4 Debugging Techniques	Trace tables, breakpoints, print statements, syntax/runtime/logic errors

B2.1.1 Variables and Data Types

Core Concepts

Concept	Description
Variable	A named storage location in memory that holds a value
Global variable	Declared outside any function; accessible throughout the program
Local variable	Declared inside a function; only accessible within that function
Scope	The region where a variable is accessible

Data Types in Python

Data Type	Description	Example
int	Whole numbers	<code>age = 25</code>
float	Decimal numbers	<code>price = 19.99</code>

Data Type	Description	Example
bool	True or false	is_passed = True
str	Sequence of characters	name = "Alice"

Global vs. Local Variables

```
python

# Global variable
counter = 0

def increment():
    global counter # Required to modify global variable
    counter += 1

def main():
    # Local variable
    local_num = 5
    print(local_num)
    increment()
    print(counter) # Accessible

main()
# print(local_num) # NOT accessible (local to main)
```

Trace Table Example

```
python

total = 0
def main():
    x = 5
    y = 10
    total = x + y
    print(total)
```

Line	x	y	total	Output
1 (start)	-	-	0	-
2 (x = 5)	5	-	0	-
3 (y = 10)	5	10	0	-
4 (total = x + y)	5	10	15	-
5 (print)	5	10	15	15

B2.1.2 String Manipulation

Core Concepts

Operation	Syntax	Example	Result
Concatenation	<code>+</code>	<code>"Hello" + " " + "World"</code>	<code>"Hello World"</code>
Slicing	<code>[start:end]</code>	<code>"Hello"[0:2]</code>	<code>"He"</code>
Replacement	<code>.replace()</code>	<code>"cats".replace("cats", "dogs")</code>	<code>"dogs"</code>

Key Principles

text

STRING MANIPULATION PRINCIPLES

- ✔ Strings are 0-indexed: first character at index 0
- ✔ Strings are immutable: operations create new strings
- ✔ Slicing: start inclusive, end exclusive
- ✔ `.replace()` returns a new string

Examples

python

```
# Concatenation
first = "Bob"
last = "Smith"
full = first + " " + last # "Bob Smith"

# Slicing
greeting = "Hello World"
word = greeting[0:5] # "Hello"

# Replacement
msg = "I like cats"
new_msg = msg.replace("cats", "dogs") # "I like dogs"

# Slicing + concatenation
name = "Hello"
new_name = "J" + name[1:] # "Jello"

# Extract username from email
email = "user123@gmail.com"
at_index = email.find('@')
username = email[:at_index] # "user123"
domain = email[at_index + 1:] # "gmail.com"
```

B2.1.3 Exception Handling

What is Exception Handling?

Exception handling is the process of responding to errors (exceptions) that occur during program execution to prevent crashes and ensure robust behaviour.

Common Exceptions in Python

Exception	Cause
<code>NameError</code>	Using a variable that hasn't been defined
<code>TypeError</code>	Using an operation on the wrong data type
<code>ValueError</code>	Passing an inappropriate value to a function
<code>ZeroDivisionError</code>	Dividing by zero
<code>IndexError</code>	Accessing an index out of range
<code>FileNotFoundError</code>	File not found when opening

try-except Syntax

```
python

try:
    # Code that might cause an error
    num = int(input("Enter a number: "))
    result = 10 / num
    print(result)
except ValueError:
    print("Please enter a valid number.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
except Exception as e:
    print(f"An error occurred: {e}")
finally:
    print("This always executes.")
```

Example: Safe Division

```
python

def safe_divide():
    try:
        numerator = float(input("Enter numerator: "))
        denominator = float(input("Enter denominator: "))
        result = numerator / denominator
        print(f"Result: {result}")
    except ValueError:
        print("Please enter valid numbers.")
    except ZeroDivisionError:
        print("Cannot divide by zero.")
    finally:
        print("Division attempt completed.")

safe_divide()
```

B2.1.4 Debugging Techniques

What is Debugging?

Debugging is the process of identifying and fixing errors (bugs) in a program.

Types of Errors

Error Type	Description	Example
Syntax Error	Violation of language rules	Missing colon, unmatched parentheses
Runtime Error	Occurs during execution	Division by zero, file not found
Logic Error	Program runs but produces incorrect results	Wrong formula, incorrect condition

Debugging Techniques

Technique	Description
Trace Tables	Track variable values step by step
Print Statements	Insert <code>print()</code> to check variable values
Breakpoints	Pause execution at specific lines
Step-by-Step Execution	Run code line by line
Error Message Analysis	Read and interpret error messages

Debugging Example

```
python

# Buggy program
def calculate_average(numbers):
    total = 0
    for num in numbers:
        total += num
    return total / len(numbers) # Potential bug: division by zero

# Using print statements to debug
def debug_average(numbers):
    print(f"Input: {numbers}")
    total = 0
    for num in numbers:
        total += num
    print(f"Total: {total}")
    print(f"Length: {len(numbers)}")
    return total / len(numbers)

# Test with empty List
# debug_average([]) # Would cause ZeroDivisionError
```

Error Message Analysis

```
text

Traceback (most recent call last):
  File "example.py", line 10, in <module>
    result = 10 / num
ZeroDivisionError: division by zero

# Interpretation:
```

```
# - File: example.py
# - Line: 10
# - Error Type: ZeroDivisionError
# - Cause: Attempted to divide by zero
```

Lesson Resources

Presentations

Topic	Link
Variables and Data Types	Presentation
String Manipulation	Presentation
Exception Handling	Presentation
Debugging Techniques	Presentation

Worksheets

Topic	Worksheet	Answers
Variables and Data Types	B2.1.1 Worksheet	B2.1.1 Answers
String Manipulation	B2.1.2 Worksheet	B2.1.2 Answers
Exception Handling	B2.1.3 Worksheet	—
Debugging Techniques	B2.1.4 Worksheet	B2.1.4 Answers

Summary: Key Takeaways

Topic	Key Point
Variables	Store data; scope determines accessibility
Data Types	int, float, bool, str
String Manipulation	Concatenation (+), slicing ([::-1]), replacement (.replace())
Exception Handling	try-except blocks prevent crashes
Debugging	Trace tables, print statements, breakpoints

text

KEY REMINDERS	
✔	Variables store data; local vs. global scope
✔	Data types: int, float, bool, str
✔	Strings are 0-indexed and immutable
✔	Exception handling prevents crashes

✓ Debugging techniques find and fix errors

"Programming fundamentals—variables, data types, string manipulation, exception handling, and debugging—are the building blocks of all software development."